# Jedi Pack training page

Official course (free with the hackathon, sign in with "Continue with Hackathon"): https://academy.genai.works/courses/open-agent-hackathon-2026

9 lessons in 4 modules. All read on 18 Oct 2026. Recordings for the live sessions appear in each lesson after the session.

Each lesson has three summaries.

- **20%**: the working version. Enough to do the task without watching.
- **4%**: the key points, about a minute.
- **1%**: one line.

## Read this first: what changes our plan

| Finding | Source | What we do |
|---|---|---|
| Product must run on **Nemotron through Meterless**, not NVIDIA direct | 1.1, 4.1 | Add METERLESS_BASE_URL / METERLESS_API_KEY / NEMOTRON_MODEL path in `llm.mjs`; NVIDIA direct stays as practice fallback |
| **Cursor is mandatory** for development, and the submission must explain how it was used | 1.2, 3.1 | Use Cursor for the contest build and say so in the README. Other AI tools (Gemini, Claude, Codex) are fine for practice and planning; the rules say any model is allowed and generated code is expected |
| Cursor credit and Meterless credit are claimed on the **HackOS task board** | 3.1, 4.1 | Paul or a team member claims them at hackathon.genai.works once the team is set |
| **Prize pool is $15,000 cash + $5,000 in-kind** (not $13k + $7k) | 1.2 | Per-track cash: 1st $1,500-1,600, 2nd $1,200, 3rd $900. Wildcard $1,000 / $600 / $400. Update the split discussion with these numbers |
| Work committed before 22 Oct 00:00 UTC is not eligible **unless declared** as pre-existing in the submission form | 1.2 | Declare the Gift Finder if we reuse any of it. The Wildcard track is built for bring-your-own-project and is judged on the extension |
| Zetaris and Meterless must both be integrated, and the video, deck and README must show them | 1.2 | Plan the demo around visible Zetaris queries |
| Submission is made in **HackOS**: https://hackathon.genai.works/dashboard | 1.2 | Add to the team playbook |
| Dates: onboarding call **21 Oct 12:00 UTC** (YouTube live); build window 22 Oct 00:00 to 27 Oct 23:45 UTC; results 6 Nov | 1.2, 1.3 | Earlier notes said 12 Oct for the call; that was wrong or was the recording |
| Workshops: Power Day 19 Oct 3 PM PST, Build Day 20 Oct 9 PM PST, Data Day listed as 12 Oct 3 PM PST (recording) | 2.2, 3.2, 4.2 | Calendar the first two |

| Finding | Source | What we do |
| --- | --- | --- |
| Organisers still owe: Zetaris image access link, Meterless base URL, Meterless "AI hotspot" description | 1.1, 4.1 | Ask in the Discord help channel |

---

## 1.1 Start Here: set up your Data, Build and Power layers

**20%**

- **Three required layers:** Data = Zetaris, Build = Cursor, Power = NVIDIA Nemotron served through Meterless.
- **Credits:** Cursor credit pays while you code. Meterless credit pays only when the product runs. Keep the Meterless key out of Cursor's settings.
- **Zetaris install (public Freemium):** request image access first (up to 24 h). `git clone https://github.com/zetaris/Freemium.git`, `docker login`, then pull and start only four containers: zetaris-db, zetaris-server, zetaris-api, zetaris-gui. Create a login with the `generate_default_dev_users.sh` script, open http://localhost:9001. Docker needs 8 GB+ memory (16 GB laptop).
- **Ports:** UI 9001, REST API 8888 (the one your agent uses), JDBC 10000.
- **Meterless wiring:** OpenAI-format client with `METERLESS_BASE_URL`, `METERLESS_API_KEY`, `NEMOTRON_MODEL` in `.env`. `.env` must be in `.gitignore`.
- **Starter layout:** `agent/llm.py`, `agent/tools_zetaris.py` (a `run_sql` function), `main.py` (the loop), and `smoke_test.py` which prints three PASS lines.
- **Agent loop:** Nemotron decides, your code runs `run_sql` against Zetaris, result goes back, repeat.

**4%**

- Zetaris for data, Cursor to build, Nemotron via Meterless to run.
- Two credit pots, two keys, never mix them.
- Start the Zetaris install first because access takes up to a day.

**1%**

Data in Zetaris, build in Cursor, run on Nemotron through Meterless.

---

## 1.2 How the week runs: tracks, rules, judging, submission

**20%**

- **Dates:** registration closes 22 Oct; onboarding call 21 Oct; build window 22 Oct 00:00 UTC to 27 Oct 23:45 UTC; results 6 Nov. All deadlines UTC.
- **Tracks (pick one):** 01 Solving Fragmented Intelligence (many sources, Zetaris), 02 The Agent That Can Explain Why (evidence-backed answers), 03 Reasoning Architecture (data, memory, reasoning to

decisions), 04 Wildcard Tinkerer (bring your own project, extend it with sponsor tech). You can change track until the deadline.
- **Must use:** Zetaris and Meterless in every submission, Cursor for development. Use NVIDIA where it helps; running on Nemotron through Meterless covers it. Any language, framework, model or cloud allowed. Generated code is allowed and expected.
- **Pre-existing work:** committed before the window opens is not eligible except clearly declared pre-existing components. Open-source libraries are fine; product code must be declared in the submission form.
- **Teams:** 1 to 5 people, aged 16+, one team per person, composition locks when the first submission is created. Duplicate accounts disqualify the team.
- **Judging, 100 points:** Impact 30, Technical 20, Innovation 15, Demo 15, Product UX 10, Sponsor tech 10.
- **Prizes:** up to $20,000: $15,000 cash plus $5,000 in-kind. Tracks 01 and 02: $1,600 / $1,200 / $900. Track 03: $1,500 / $1,200 / $900. Wildcard: $1,000 / $600 / $400.
- **Submit four things in HackOS:** 1 to 4 minute demo video, slide deck (with architecture diagram), GitHub repo with README, and a clear account of how Zetaris, Meterless, Cursor and NVIDIA were used.
- **Checklist:** track selected; video 1 to 4 min and end to end; video and deck name Zetaris, Meterless and Cursor; README lets an outsider run it; no secrets in the repo; pre-existing code declared; submitted before 23:45 UTC on 27 Oct.
- You keep all IP; organisers get a licence to show demo, screenshots and description.

**4%**

- Pick one of four tracks; must use Zetaris, Meterless and Cursor.
- Impact (30) and Demo (15) together are nearly half the score.
- Four deliverables, submitted in HackOS by 27 Oct 23:45 UTC.
- Anything built before 22 Oct must be declared.

**1%**

Solve a real problem with Zetaris + Meterless + Cursor, show it clearly, submit by 27 Oct.

---

## 1.3 Onboarding call

**20%**

- Live on YouTube **21 Oct 2026, 12:00 UTC**; recording appears in the lesson afterwards.
- Covers how the week runs, the tracks, what a strong submission looks like (a production-oriented agent that reaches real data, reasons across steps, keeps context and turns understanding into an outcome), team rules, dates, judging, deliverables and the prize pool.
- The lesson itself is a description only, so watch the recording for anything not in 1.2.

**4%**

A walkthrough of the rules already summarised in 1.2, with Q&A.

**1%**

Watch it for the Q&A, the rules are in 1.2.

---

## 2.1 Data Quick Start: Zetaris

**20%**

- **Why Zetaris:** one access point to many sources (databases, JSON, REST, PDFs), no connector per source, a semantic layer for grounding answers, and it is the headline sponsor.
- **Three routes:** web UI (9001) to explore and test SQL by hand, REST API (8888) for your agent, JDBC (10000) for tools. The API docs describe the hosted service, so check differences for the local install with mentors.
- **Zetaris starts empty.** Register sample data: knowledge base article "How do I connect to test data sources?", run CREATE DATASOURCE and REGISTER DATASOURCE TABLES in the SQL Editor (Schema Store View). The Azure Postgres "sports sample data" source is a good first pick.
- **Your own data:** put a CSV in `Freemium/customer_data`, register it, join it with other data in one query. JDBC drivers go in `customer_data/lib/jdbc` (restart zetaris-server).
- **Pattern that scores:** ground every answer in a query the user can see (track 02); land every source in Zetaris and join in one SQL query (track 01). Example: weather CSV joined with customer locations to decide where to sell umbrellas.
- **Troubleshooting:** "denied/unauthorized" means access not active or wrong Docker Hub account; nvidia driver error means you started the GPU service, start only the four core ones; "Failed to connect to zetaris-server:10000" is normal for a couple of minutes; Windows `\r` errors need `windows_convert.sh`; "port already allocated" means another program (e.g. Postgres on 5432) is using it.
- Before the workshop: four containers up, a table returns rows, and one business question that needs two or more sources.

**4%**

- Put Zetaris at the centre, make every answer trace to a query.
- Use two or more sources joined in one query.
- Keep a hand-run query in the UI as your baseline.

**1%**

Zetaris is the one place your agent asks for data; show the queries.

---

## 2.2 Workshop: Data Day with Zetaris

**20%**

- Hands-on live session on connecting an agent to Zetaris via the REST API (8888), running a first query, and querying more than one source in one request.

- Listed as 12 Oct, 3 PM PST (22:00 UTC); recording appears in the lesson.
- Come with four containers running and a table returning rows. It feeds tracks 01 and 02 and the Impact and Sponsor Tech scores.

**4%**

Watch the recording once Zetaris is running; do the multi-source query yourself.

**1%**

Practical session on the REST API and cross-source queries.

---

## 3.1 Build Quick Start: Cursor and the Meterless prompt library

**20%**

- **Build versus run:** Cursor's own models (Composer for fast everyday coding, Grok for harder longer tasks) do the coding on Cursor credit, no API key. Nemotron through Meterless powers the product on Meterless credit.
- **Set up Cursor:** download from cursor.com and sign in. **Claim Cursor credit on the HackOS task board** (hackathon.genai.works, create or join a team, open the board).
- **Check:** the Agent creates and runs a hello file with Composer or Grok and no key entered.
- **Meterless prompt library:** free prompt files at meterless.ai/engines.html (agent scaffold, memory and similar). Save in `prompts/`, hand to the Agent with `@filename`. Note in the README which engines you used and why.
- **Vibe coding in three moves:** Describe (starter layout and scaffold prompt), Review (read generated code, secrets from `.env`, nothing hard-coded), Steer (one feature at a time, give errors not guesses, commit after each working step).
- **Judges look for:** a "How we built it" README section naming Cursor, the models used and Meterless engines; setup that works in five minutes.
- **Troubleshooting:** Grok or Composer missing means credit inactive or used up; Meterless charges while coding means the key is in Cursor settings; the Agent can't run a file means Python is not on PATH.

**4%**

- Code with Cursor models, run with Meterless.
- Use the prompt library and say so in the README.
- Commit after each working step.

**1%**

Build in Cursor on Cursor credit, run on Meterless credit, document both.

---

## 3.2 Workshop: Build Day, vibe coding with Cursor and Meterless

**20%**

- Live build of a small agent from an empty folder: scaffolds `llm.py` (Nemotron through Meterless), `tools_zetaris.py` (`run_sql`) and `main.py` (the loop). Covers choosing Composer or Grok, handing prompt files with `@`, reviewing and steering, and keeping secrets in `.env`.
- Listed as Tuesday 20 Oct, 9:00 PM PST (check the UTC time, the lessons are inconsistent about PST/PDT); recording appears after.
- Backbone of the Technical score (20 points).

**4%**

Watch it live or on the recording; follow along with your own layers set up.

**1%**

Empty folder to running agent loop in one sitting.

---

## 4.1 Power Quick Start: Meterless and Nemotron

**20%**

- **Two providers, one inside the other:** your app calls Meterless, Meterless runs Nemotron. This satisfies both "how Meterless was integrated" and "where NVIDIA contributes".
- **Setup:** claim Meterless credit on the HackOS task board, create an API key in Meterless, put `METERLESS_API_KEY`, `METERLESS_BASE_URL` and `NEMOTRON_MODEL` (the exact model id Meterless shows) in `.env`. The base URL and the "AI hotspot" description were still "[ORGANISERS TO ADD]".
- **Models:** Super is the default for the main agent, Nano for fast helper steps (classify, summarise, format), Ultra for deep planning where a wrong plan is expensive. You can mix them.
- **Code:** the standard `openai` package, `base_url` from `METERLESS_BASE_URL`, key from `METERLESS_API_KEY`, `model` from `NEMOTRON_MODEL`. Check 3 prints "PASS Power: Nemotron replies through Meterless".
- **Agent loop cost:** every loop iteration is a paid call. Tool descriptions are prompts. Plan first rather than looping ten times.
- **Cost discipline:** Nano for helpers, Super for main, Ultra for planning only; cap iterations per request; test with a few real questions; keep the Meterless key out of Cursor; check remaining credit the day before submission.
- **Troubleshooting:** missing setting means check `.env` location; 401 wrong key; 404 or "model not found" wrong model id; credit or rate-limit error means credit out or throttled.

**4%**

- Super for the main agent, Nano for helpers, Ultra for planning.
- Cap loop iterations and test sparingly.

- Check credit before recording the demo.

**1%**

Run on Nemotron through Meterless and keep the loop cheap.

---

## 4.2 Workshop: Power Day with Meterless and Nemotron

**20%**

- Live session on setting up Meterless with Nemotron, wiring it into the agent loop, choosing Super, Nano or Ultra, where token cost and latency come from, tool calling (describing `run_sql`), and guardrails (iteration caps, clear stops, human escalation).
- Listed as Monday 19 Oct, 3:00 PM PST (check the UTC time, the lessons are inconsistent about PST/PDT); recording appears after.
- Backbone of track 03, Reasoning Architecture.

**4%**

Watch for cost control and tool calling tips.

**1%**

Cheap, reliable agent loops on Nemotron.